



线程的创建（上）

北京理工大学计算机学院
金旭亮

三种创建线程的方法



1

从Thread类派生出子类

2

使用Runnable封装线程函数

3

使用线程工厂

实现多线程运行代码的基本方法（之一）

- ▶ 可以直接从Thread类派生，重写其run方法（此即“线程函数”）：

```
class MyThread extends Thread {  
    public void run() {  
        //将要以多线程方式运行的代码  
    }  
}
```

- ▶ 调用Thread类的start方法启动运行。



start()方法只能调用一次，多次调用，会抛出IllegalThreadStateException异常。

ThreadRun.java示例



```
ThreadRun.java x
1 public class ThreadRun extends Thread {
2     public void run() {
3         System.out.println("辅助线程名: " + Thread.currentThread().getName());
4         for (int i = 0; i < 10; i++)
5             System.out.println("Thread counter=" + i);
6     }
7
8     public static void main(String[] args) {
9         System.out.println("主线程名: " + Thread.currentThread().getName());
10        ThreadRun th = new ThreadRun();
11        th.start();
12    }
13 }
```



这种编程方式造成了代码间的紧耦合，不推荐使用。

ThreadRun.java示例



运行结果:

```
Run: ThreadRun x
"C:\Program Files\Java\jdk-17\bin\java.exe"
主线程名: main
辅助线程名: Thread-0
Thread counter=0
Thread counter=1
Thread counter=2
Thread counter=3
Thread counter=4
Thread counter=5
Thread counter=6
Thread counter=7
Thread counter=8
Thread counter=9

Process finished with exit code 0
```

主线程与辅助线程并发执行。

主线程有一个“main”的名字，其它的线程名格式为“Thread-序号”。



线程对象的名字是如何起的？

Thread类在内部定义了一个只增的int字段，用于构造“匿名线程”的Name属性，可以使用getName()方法获取这个名字。

```
/* For autonumbering anonymous threads. */  
private static int threadInitNumber;  
  
private static synchronized int nextThreadNum() {  
    return threadInitNumber++;  
}
```



请连续启动多个线程，观察未指定情况下线程的名字。

可以直接指定线程的名字

Thread类定义有以下的构造方法，可以直接在创建线程对象时，给它指定一个名字：

```
Thread (String name)
```

示例：

```
class MyNamedThread extends Thread {  
    1 usage  
    public MyNamedThread(String name) {  
        super(name);  
    }  
  
    @Override  
    public void run() {  
        System.out.println(getName() + "正在工作。");  
    }  
}
```

SetThreadName.java

使用自定义名字的线程



```
SetThreadName.java x
1  public class SetThreadName {
2      public static void main(String[] args) {
3          new MyNamedThread("我的线程").start();
4      }
5  }
6
7  class MyNamedThread extends Thread {...}
```

1 usage

```
Run: SetThreadName x
"C:\Program Files\Java\jdk-17\bin\java.exe"
我的线程正在工作。
Process finished with exit code 0
```


实现多线程运行代码的基本方法之二 (1)

将代码封装到Runnable接口的run方法内：

```
public interface Runnable {  
    void run();  
}
```



```
class MyRunnable implements Runnable {  
    public void run() {  
        //要以多线程方式运行的代码  
    }  
}
```

实现多线程运行代码的基本方法之二 (2)

- ▶ 创建一个Runnable对象:

```
Runnable r = new MyRunnable();
```

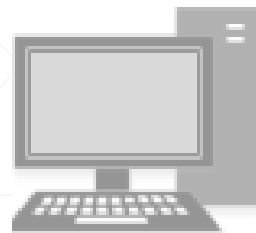
- ▶ 将Runnable对象传给Thread对象:

```
Thread t = new Thread(r);
```

- ▶ 启动多线程运行:

```
t.start();
```

RunnableThreadTest.java示例



//使用MyThread类封装线程要执行的代码

```
class MyThread implements Runnable {  
    public void run() {  
        var threadName : String = Thread.currentThread().getName();  
        System.out.println("辅助线程" + threadName + "已经启动运行");  
        for (int i = 0; i < 10; i++)  
            System.out.println("Thread counter=" + i);  
        System.out.println("线程" + threadName + "运行结束");  
    }  
}
```

```
public class RunnableThreadTest {  
    public static void main(String[] args) {  
        var threadName : String = Thread.currentThread().getName();  
        System.out.println("主线程名: " + threadName);  
        //创建新线程并运行它  
        Thread th = new Thread(new MyThread());  
        th.start();  
        System.out.println("主线程" + threadName + "运行结束");  
    }  
}
```

RunnableThreadTest.java

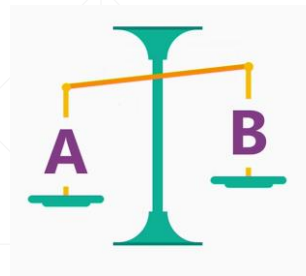


```
Run: RunnableThreadTest x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
主线程名: main
主线程main运行结束
辅助线程Thread-0已经启动运行
Thread counter=0
Thread counter=1
Thread counter=2
Thread counter=3
Thread counter=4
Thread counter=5
Thread counter=6
Thread counter=7
Thread counter=8
Thread counter=9
线程Thread-0运行结束

Process finished with exit code 0
```

主线程运行结束了，辅助线程还在运行。
只有当两个线程都运行完毕时，
进程才算终止。

两种创建线程方式的对比



从Thread派生出子类的方式实现多线程，其长处在于可以为此线程类方便地定义各种字段和私有方法，更为模块化，适用于封装特定的数据处理任务，但不够灵活（因为Java不支持多继承，一个类只能有一个父类）。



使用Runnable接口方式实现多线程，可以把“线程”和“任务”分离，因此这是一种推荐的编程模式。

使用Lambda表达式简化线程的创建



```
LambdaThreadTest.java x
1  public class LambdaThreadTest {
2      public static void main(String[] args) {
3          Thread lambdaThread = new Thread(() -> {
4              System.out.println("使用Lambda创建的Thread对象:"
5                  + Thread.currentThread().getName());
6          });
7          lambdaThread.start();
8      }
9  }
```



在实际开发中，既可以使用Lambda表达式，也可以使用Runnable对象实现线程，但Lambda表达式更简洁，所以现在用得越来越多。

获取当前执行线程



如果你需要知道正在负责执行某段代码的线程是哪一个，可以调用 Thread 类所提供的静态方法 `currentThread()` 获取当前线程的引用。

```
CurrentThread.java x
1 public class CurrentThread {
2     public static void main(String[] args) {
3         //输出: main
4         showCurrentThread();
5         //输出: Thread-0
6         new Thread(CurrentThread::showCurrentThread).start();
7     }
8
9     static void showCurrentThread(){
10        System.out.println(Thread.currentThread().getName());
11    }
12 }
```

巩固练习



主线程启动一个工作线程，这个工作线程在运行过程中调用了另一个函数，那这个被调用的函数运行于哪个线程中？编写代码验证你的结论。